OPERATING SYSTEMS

UNIT IV – MEMORY MANAGEMENT STRATEGIES & MASS STORAGE

DVR & Dr. HS MIC College of Technology (Autonomous)
All Section B, Section C and Section D Questions

Answer format: Easy English, what-it-is introduction, explanation, steps, examples, block diagrams, advantages, disadvantages and conclusion. Numerical problems include step-by-step working, tables and final results.

UNIT IV – MEMORY MANAGEMENT STRATEGIES & MASS STORAGE

SECTION B – 5-MARK LONG ANSWER QUESTIONS (EXPANDED INTO 10-MARK FORMAT)

Q1. Discuss Demand Paging and describe Page Fault Handling.

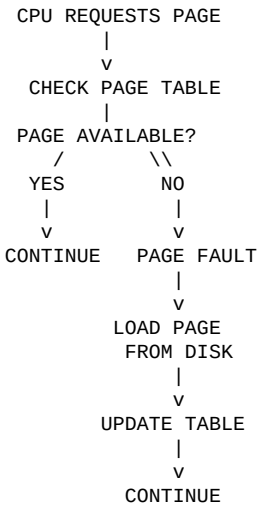
1. What is Demand Paging?

Demand Paging is a memory management technique in which pages are loaded into RAM only when they are needed. It does not load the entire program into memory. It loads only the required pages and saves main memory.

Example: A program has 100 pages, but currently needs only 10 pages. The OS loads only those 10 pages.

2. Working of Demand Paging

1. A program starts execution.
2. CPU requests a particular page.
3. OS checks the page table.
4. If the page is available in RAM, execution continues.
5. If the page is missing, a Page Fault occurs.
6. OS loads the required page from disk.
7. The page table is updated and the process continues.



3. What is a Page Fault?

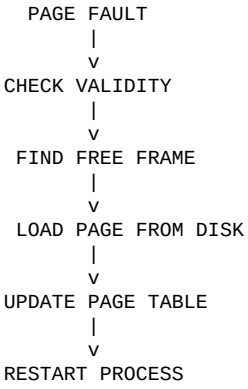
A Page Fault occurs when a process requests a page that is not available in RAM. Example: if the CPU requests Page 5 but Page 5 is not in RAM, a Page Fault occurs.

4. Steps in Page Fault Handling

1. CPU requests a page.
2. OS checks the page table.
3. OS identifies that the page is missing.
4. A Page Fault trap occurs.
5. OS checks whether the requested memory access is valid.
6. OS finds a free frame. If no frame is free, it selects a page for replacement.
7. The required page is loaded from disk into RAM.

8. The page table is updated.

9. The interrupted instruction is restarted.



5. Advantages

• Saves RAM.

• Loads only required pages.

• Allows large programs to execute.

• Improves memory utilization.

• More processes can stay in memory.

6. Disadvantages

• Page faults take extra time.

• Disk access is slower than RAM.

• Too many page faults cause thrashing.

7. Conclusion

Demand Paging loads pages only when they are required. Page Fault Handling brings missing pages from disk into RAM and allows the process to continue.

Remember: Demand Paging = Load When Needed. Page Fault = Page Not Available in RAM.

Q2. Explain Contiguous Memory Allocation and discuss fragmentation issues (internal and external).

1. What is Contiguous Memory Allocation?

Contiguous Memory Allocation is a memory management technique in which each process receives one continuous block of memory. The entire process is stored in one connected memory area. Example: if a process needs 100 KB, the OS allocates one continuous 100 KB memory block.

2. Working

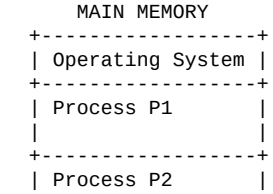
1. OS checks available memory.

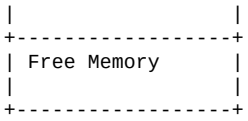
2. It finds a suitable free memory block.

3. It allocates the block to the process.

4. The process executes in that memory.

5. After execution, memory is released.





3. Types of Contiguous Allocation

A. Fixed Partitioning: Memory is divided into fixed-size partitions. Each partition can hold one process. Partition size cannot change.

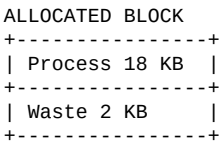
B. Variable Partitioning: Partitions are created according to process requirements. Memory size is allocated dynamically. It uses memory more flexibly.

4. What is Fragmentation?

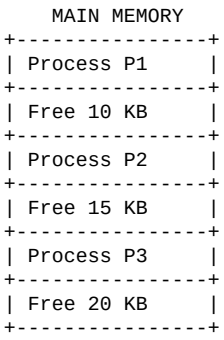
Fragmentation means wastage of memory space. There are two main types.

A. Internal Fragmentation: Occurs when allocated memory is larger than the memory required by a process. The unused space is inside the allocated block.

Example: Allocated memory = 20 KB; required memory = 18 KB; wasted memory = 2 KB.



B. External Fragmentation: Occurs when free memory is divided into small separate blocks. Total free memory may be enough, but a large continuous block is not available.



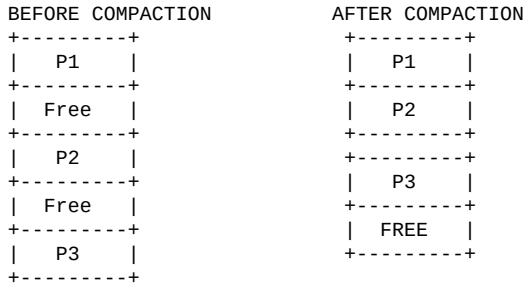
Example: Total free memory = 45 KB. A process requiring 40 KB cannot fit into any single free block.

5. Methods to Reduce Fragmentation

1. Compaction: Combines small free spaces into one large continuous block.

2. Paging: Divides memory into pages and frames to avoid external fragmentation.

3. Better Allocation: Selects suitable memory blocks for processes.



6. Advantages

- Easy to understand.
- Simple memory allocation.
- Fast memory access.
- Easy address calculation.

7. Disadvantages

• Internal fragmentation may occur.

• External fragmentation may occur.

• Large continuous memory is difficult to find.

• Compaction may take extra time.

8. Conclusion

Contiguous Memory Allocation gives each process one continuous memory block. It is simple, but fragmentation can waste memory.

Remember: Internal = Waste inside allocated block. External = Free spaces separated outside blocks.

Q3. What is Translation Lookaside Buffer (TLB)? Explain its working with a diagram.

1. What is TLB?

TLB stands for Translation Lookaside Buffer. It is a small and fast memory that stores recently used page table entries. Its main purpose is to make address translation faster.

Example: Instead of checking the page table every time, the CPU first checks the TLB.

2. Why is TLB Required?

• Page tables are stored in main memory.

• Accessing the page table takes extra time.

• TLB stores recently used entries.

• It reduces memory access time.

3. Working of TLB

1. CPU generates a virtual address.

2. MMU checks the TLB.

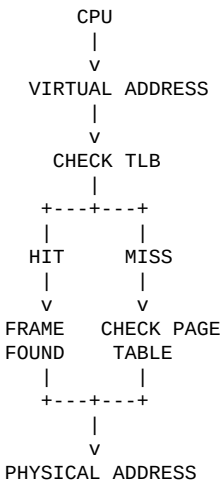
3. If the page number is found, it is called a TLB Hit.

4. If the page number is not found, it is called a TLB Miss.

5. During a miss, the page table in RAM is checked.

6. The required entry is added to TLB.

7. Physical address is generated.



4. Types of TLB Results

TLB Hit: The required page entry is available in TLB. Access is faster and there is no need to check the page table in RAM.

TLB Miss: The required entry is not available in TLB. The page table must be checked and the entry can be added to TLB.

5. TLB Hit Ratio

Hit Ratio represents how often the required page entry is found in TLB.

Hit Ratio = TLB Hits / Total Accesses

Example: 100 memory accesses and 80 TLB hits: Hit Ratio = 80/100 = 0.8 = 80%.

6. Advantages

• Faster address translation.

• Reduces memory access time.

• Improves CPU performance.

• Reduces repeated page table access.

7. Disadvantages

• TLB has limited storage.

• TLB misses require additional access.

• TLB hardware increases system cost.

8. Conclusion

TLB is a fast memory that stores recently used page table entries. It reduces address translation time and improves system performance.

Remember: TLB = Fast Page Table Storage.

Q4. Explain FCFS and SSTF disk scheduling algorithms with examples.

1. What is Disk Scheduling?

Disk Scheduling is a method used by the Operating System to decide the order of disk requests. Its main purpose is to reduce disk-head movement and waiting time.

2. FCFS (First-Come, First-Served)

FCFS serves disk requests in the order in which they arrive.

Working

1. Requests enter the queue.

2. The disk head takes the first request.

3. It completes that request.

4. It moves to the next request.

5. This continues until all requests are completed.

Example: Head position = 50; Requests = 40, 20, 60.

$50 \rightarrow 40 \rightarrow 20 \rightarrow 60$

Calculation: (50−40) + (40−20) + (60−20) = 10 + 20 + 40 = 70 cylinders.

FCFS Advantages and Disadvantages

• Advantages: Simple to understand; easy to implement; no starvation due to request ordering.

• Disadvantage: Head movement may be high and performance may be slow.

3. SSTF (Shortest Seek Time First)

SSTF selects the disk request nearest to the current head position.

Working

1. Check all pending requests.

2. Find the nearest request.

3. Move the head to that request.

4. Remove the completed request.

5. Repeat until all requests are completed.

Example: Head position = 50; Requests = 40, 20, 60. The nearest request is 40 or 60 (both are 10 cylinders away). Select 40. From 40, nearest is 20. Then serve 60.

```
50 → 40 → 20 → 60
```

Calculation: (50−40) + (40−20) + (60−20) = 10 + 20 + 40 = 70 cylinders.

SSTF Advantages and Disadvantages

• Advantages: Reduces head movement; often performs better than FCFS.

• Disadvantage: Far requests may wait for a long time.

4. Comparison

FCFS	SSTF
Follows arrival order	Selects nearest request
Simple method	More selection work
Head movement can be high	Usually reduces movement
No starvation due to ordering	Starvation is possible

5. Conclusion

FCFS serves requests in arrival order, while SSTF serves the nearest request first. Both methods are used to manage disk requests efficiently.

Remember: FCFS = First request first. SSTF = Nearest request first.

Q5. What is Paging? Draw and explain the structure of a Page Table.

1. What is Paging?

Paging is a memory management technique that divides a process into fixed-size blocks called Pages. Main memory is divided into equal-size blocks called Frames. Page size and frame size are always equal.

Example: If a process contains 4 pages, the OS can store those pages in any 4 available frames.

2. Important Terms

• Page: Small fixed-size block of a process.

• Frame: Small fixed-size block of RAM.

• Page Table: Stores the mapping between pages and frames.

• Page Number: Identifies a page.

• Frame Number: Identifies a frame.

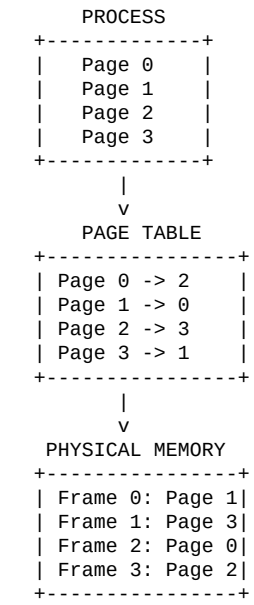
3. Structure of a Page Table

A Page Table is a data structure maintained by the Operating System. It stores the frame number corresponding to each page.

Page Number	Frame Number
Page 0	2
Page 1	0
Page 2	3

Page Number	Frame Number
Page 3	1

This means Page 0 is stored in Frame 2.



4. Working of Paging

1. A process is divided into pages.

2. RAM is divided into frames.

3. OS finds available frames.

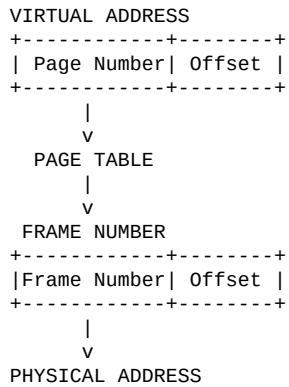
4. Pages are loaded into those frames.

5. Page Table stores page-to-frame mapping.

6. CPU uses the Page Table to access the required page.

5. Address Translation

A Virtual Address contains a Page Number and an Offset. The page number is converted into a frame number using the page table. The frame number and offset form the physical address.



6. Advantages

• Avoids external fragmentation.

• Memory allocation is flexible.

• Pages can be stored in different frames.

• Supports virtual memory.

• Improves memory utilization.

7. Disadvantages

• Page tables require extra memory.

• Address translation takes time.

• Internal fragmentation may occur in the last page.

8. Conclusion

Paging divides processes into pages and RAM into frames. The Page Table connects pages with frames and helps the CPU access the correct memory location.

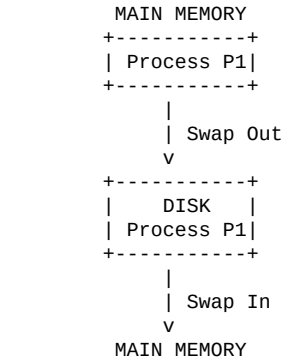
Remember: Page = Process block; Frame = RAM block; Page Table = Mapping.

Q6. Differentiate between Swapping and Paging with suitable examples.

1. What is Swapping?

Swapping is a memory management technique in which an entire process is temporarily moved from RAM to secondary storage. When required, the process is brought back into RAM.

Example: A process is temporarily moved from RAM to disk to make space for another process.



2. Working of Swapping

1. OS selects a process.

2. The process is moved from RAM to disk.

3. Free memory becomes available.

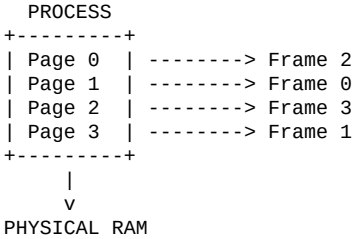
4. Another process can use that memory.

5. When required, the original process is brought back into RAM.

3. What is Paging?

Paging divides a process into fixed-size pages and RAM into equal-size frames. Only the required pages are loaded into RAM.

Example: A process has 10 pages, but currently only 4 pages are needed. The OS can load those 4 pages.



4. Difference Between Swapping and Paging

Swapping	Paging
Moves an entire process.	Moves individual pages.
Transfers larger amounts of data.	Transfers fixed-size pages.

Swapping	Paging
Process is moved to disk temporarily.	Required pages are loaded into RAM.
Usually takes more transfer time.	Transfers smaller units.
Example: Swap out Process P1.	Example: Load Page 2 into RAM.

5. Advantages of Swapping

• Frees RAM for other processes.

• Supports multiprogramming.

• Improves memory availability.

6. Advantages of Paging

• Avoids external fragmentation.

• Uses memory efficiently.

• Supports virtual memory.

• Loads only required pages.

7. Conclusion

Swapping moves an entire process between RAM and disk, whereas Paging moves individual pages. Both techniques help the Operating System manage memory.

Remember: Swapping = Whole Process; Paging = Individual Pages.

Q7. Define Virtual Memory. Explain the concept of Demand Paging.

1. What is Virtual Memory?

Virtual Memory is a memory management technique that allows a program to execute even when the entire program is not loaded into RAM. It uses secondary storage as support for main memory.

Example: A computer has 4 GB RAM, but a program needs more memory. Virtual Memory helps it run by keeping some parts on disk.

2. Why is Virtual Memory Needed?

1. Large programs may need more memory.

2. RAM is limited.

3. Multiple programs may run together.

4. Not all program pages are needed at the same time.

3. Working of Virtual Memory

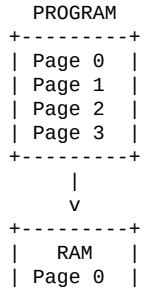
1. A program starts execution.

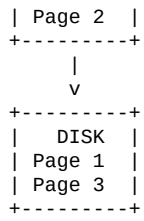
2. Required pages are loaded into RAM.

3. Other pages remain on disk.

4. When a missing page is needed, the OS loads it.

5. The program continues execution.





4. What is Demand Paging?

Demand Paging is a technique in which pages are loaded into RAM only when they are required.

Working

1. CPU requests a page.

2. OS checks the page table.

3. If the page is present, execution continues.

4. If the page is missing, a Page Fault occurs.

5. OS loads the required page from disk.

6. Page Table is updated.

7. Execution continues.

5. Demand Paging Example

Suppose a program has 100 pages. Initially, only 5 pages are loaded. CPU requests Page 20. Page 20 is not available in RAM, so a Page Fault occurs and the OS loads Page 20 from disk.

6. Advantages

• Runs large programs.

• Saves RAM.

• Supports more processes.

• Loads only required pages.

• Improves memory utilization.

7. Disadvantages

• Page faults take time.

• Disk access is slower than RAM.

• Too many page faults cause Thrashing.

8. Conclusion

Virtual Memory allows large programs to run using RAM and disk together. Demand Paging loads only the pages required during execution.

Remember: Virtual Memory = Large Logical Memory; Demand Paging = Load Required Pages.

Q8. What is Copy-on-Write? Explain its significance in memory management.

1. What is Copy-on-Write?

Copy-on-Write (COW) is a memory management technique in which two or more processes share the same memory pages until one process tries to modify a page. A separate copy is created only when a process writes to the shared page.

Example: Parent and child processes share the same memory initially. When the child changes a page, the OS creates a separate copy for the child.

2. Working of Copy-on-Write

1. Parent process creates a child process.

2. Both processes share the same memory pages.

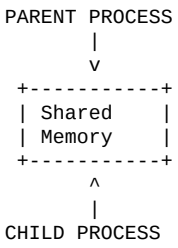
3. Initially, no separate copies are created.

4. When one process tries to modify a page, a separate copy is created.

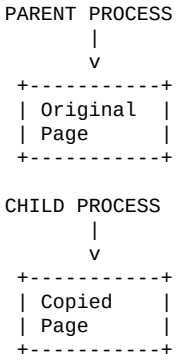
5. The process changes its own copy.

6. The other process continues using the original page.

3. Diagram – Before Modification



After Modification



4. Example

A parent process creates a child process. Both initially share 3 memory pages. The child modifies Page 1. OS creates a separate copy of Page 1. Parent continues using the original Page 1, while child modifies its copied Page 1.

5. Importance of Copy-on-Write

1. Saves memory.

2. Avoids unnecessary copying.

3. Speeds up process creation.

4. Allows processes to share memory safely.

5. Improves memory utilization.

6. Advantages

• Less memory usage.

• Faster process creation.

• No unnecessary data copying.

7. Disadvantages

• Copying takes time when a write occurs.

• Requires additional memory for copied pages.

• Page protection and fault handling are required.

8. Conclusion

Copy-on-Write allows processes to share memory until modification is required. It saves memory and improves process creation performance.

Remember: Copy-on-Write = Copy Only When Writing.

Q9. Explain the working of Page Replacement Algorithms with an example of FIFO.

1. What is Page Replacement?

Page Replacement is a memory management technique used when RAM frames are full and a new page needs to be loaded. The OS removes an existing page and replaces it with the required page.

Example: If all 3 frames are full and a new page arrives, the OS selects one existing page for replacement.

2. Why is Page Replacement Needed?

1. RAM has limited frames.

2. A process may request a new page.

3. All frames may already be occupied.

4. OS must remove one page to load the new page.

3. FIFO Page Replacement

FIFO stands for First-In, First-Out. It removes the page that entered memory first. It follows the Queue method.

4. Working of FIFO

1. Check whether the requested page is already in RAM.

2. If present, mark it as Hit (H).

3. If missing, mark it as Page Fault (PF).

4. If a free frame is available, load the page.

5. If all frames are full, remove the oldest page.

6. Load the new page.

7. Continue until all references are completed.

5. Example

Reference String: 1, 2, 3, 4. Number of Frames = 3.

Reference	Frame 1	Frame 2	Frame 3	Result
1	1	–	–	PF
2	1	2	–	PF
3	1	2	3	PF
4	4	2	3	PF

Explanation: First, pages 1, 2 and 3 are loaded. All frames become full. Page 4 arrives. Page 1 is removed because it entered first. Page 4 is loaded.

6. Advantages

• Simple to understand.

• Easy to implement.

• Uses a queue structure.

7. Disadvantages

• May remove frequently used pages.

• Can produce more page faults.

• May suffer from Belady's Anomaly.

8. Conclusion

FIFO replaces the oldest page first. It is easy to implement, but it may not always provide the best performance.

Remember: FIFO = Oldest Page Replaced First.

Q10. Define Thrashing. Why does it occur and how can it be controlled?

1. What is Thrashing?

Thrashing is a condition in which the Operating System spends more time moving pages between RAM and disk than executing processes. It happens because of frequent Page Faults.

Example: A student keeps searching for books instead of studying. Similarly, a computer keeps loading pages instead of executing programs.

2. Why Does Thrashing Occur?

1. Too many processes are running.

2. Each process gets fewer memory frames.

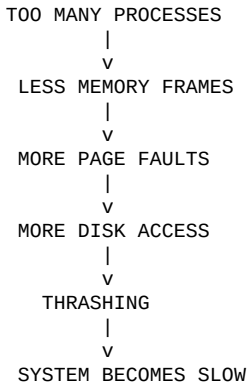
3. Required pages are not available in RAM.

4. Frequent Page Faults occur.

5. CPU spends more time loading pages.

6. System performance becomes slow.

3. Working of Thrashing



4. How to Control Thrashing?

1. Reduce Processes: Reduce the number of active processes so that enough memory is available.

2. Working Set Model: Keep the frequently required pages of a process in memory.

3. Page Fault Frequency (PFF): Monitor the number of Page Faults and allocate more frames when needed.

4. Increase Memory: Add more physical RAM if possible.

5. Advantages of Controlling Thrashing

• Improves CPU performance.

• Reduces Page Faults.

• Reduces unnecessary disk access.

• Improves system response time.

6. Conclusion

Thrashing makes the system slow because of frequent Page Faults. It can be controlled by providing enough frames and reducing the number of active processes.

Remember: Thrashing = Too Many Page Faults = Slow System.

Q11. Explain different HDD Scheduling Algorithms with an example (SCAN, C-SCAN, LOOK, C-LOOK).

1. What is HDD Scheduling?

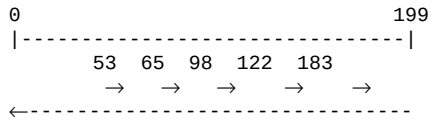
HDD Scheduling is a technique used by the Operating System to decide the order in which disk requests are served. Its main purpose is to reduce disk-head movement and waiting time.

2. SCAN Algorithm

SCAN moves the disk head in one direction, serves requests along the way and reverses direction after reaching the disk end. It is also called the Elevator Algorithm.

Working

1. Head starts at its current position.
2. It moves in one direction.
3. It serves requests along the way.
4. It reaches the disk end.
5. It reverses direction.
6. It serves remaining requests.



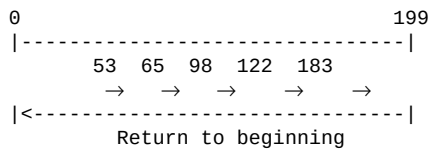
Advantages: Reduces unnecessary movement; provides better service than FCFS in many cases.
Disadvantage: It may travel to the disk end unnecessarily.

3. C-SCAN Algorithm

C-SCAN moves the disk head in only one direction. After reaching the last cylinder, it returns to the beginning and continues in the same direction.

Working

1. Head moves in one direction.
2. It serves all requests along the way.
3. It reaches the disk end.
4. It returns to the beginning.
5. It continues in the same direction.



Advantages: Provides more uniform waiting time; treats requests fairly. Disadvantage: Returning to the beginning requires extra movement.

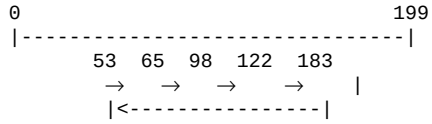
4. LOOK Algorithm

LOOK works like SCAN, but it does not travel to the physical end of the disk. It reverses direction at the last pending request.

Working

1. Head moves in one direction.
2. It serves requests along the way.
3. It reaches the last request in that direction.
4. It reverses direction.

5. It serves the remaining requests.



Advantages: Avoids unnecessary movement; reduces total head movement. Disadvantage: Waiting time may vary.

5. C-LOOK Algorithm

C-LOOK works like C-SCAN, but it moves only up to the last request. Then it jumps to the lowest pending request.

Working

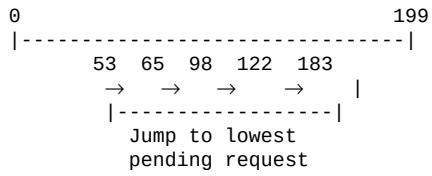
1. Head moves in one direction.

2. It serves requests along the way.

3. It reaches the highest pending request.

4. It jumps to the lowest pending request.

5. It continues in the same direction.



Advantages: Avoids unnecessary disk-end movement; provides more uniform service. Disadvantage: Circular movement requires additional head movement.

6. Comparison

Algorithm	Working
SCAN	Moves in both directions
C-SCAN	Moves in one direction and returns
LOOK	Reverses at the last request
C-LOOK	Jumps between last and first requests

7. Conclusion

SCAN, C-SCAN, LOOK and C-LOOK reduce disk-head movement by selecting an organized order for disk requests.

Remember: SCAN = Elevator. C-SCAN = Circular movement. LOOK = Reverse at last request. C-LOOK = Circular jump between requests.

Q12. Describe Disk Structure, Disk Attachment and the major activities involved in Disk Management.

1. What is Disk Structure?

A Hard Disk is a secondary storage device used to store data permanently. It contains rotating platters and different parts used for storing and accessing data.

2. Main Parts of a Hard Disk

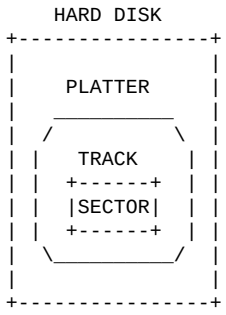
• Platter: A circular disk used to store data.

• Track: A circular path on the platter.

• Sector: A small division of a track where data is stored.

• Cylinder: A collection of tracks at the same position on different platters.

• Read/Write Head: Used to read data from and write data to the disk.



3. Disk Attachment

Disk Attachment explains how storage devices are connected to a computer.

• DAS (Direct Attached Storage): Storage directly connected to a computer. Example: Internal HDD or external USB drive.

• NAS (Network Attached Storage): Storage connected through a network. Example: Shared network storage.

• SAN (Storage Area Network): A dedicated network that provides storage access to multiple computers.

4. Major Activities in Disk Management

1. Low-Level Formatting: Prepares the disk into sectors so that data can be stored.

2. Partitioning: Divides a physical disk into separate logical sections.

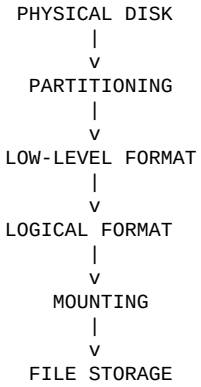
3. Logical Formatting: Creates a file system such as NTFS or FAT32.

4. Mounting: Makes a file system available for use by the Operating System.

5. Boot Management: Stores or locates the information needed to start the Operating System.

6. Bad Block Management: Identifies damaged disk blocks and prevents them from being used incorrectly.

5. Disk Management Diagram



6. Advantages

• Organizes disk storage.

• Improves data management.

• Helps protect data.

• Makes storage accessible.

• Helps identify damaged blocks.

7. Conclusion

Disk Management prepares and organizes storage devices. It helps the Operating System store, access and protect data efficiently.

Remember: Track = Circular path. Sector = Small storage unit. Cylinder = Same-position tracks. Partitioning = Dividing disk. Formatting = Preparing disk.

SECTION C – 10-MARK COMPREHENSIVE ESSAY QUESTIONS

Q1. Explain Paging Hardware, Address Translation, Translation Lookaside Buffer (TLB), and Demand Paging with Page Fault Handling.

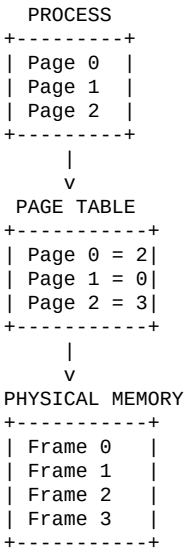
1. What is Paging?

Paging is a memory management technique used by the Operating System. It divides a process into small fixed-size blocks called Pages. It divides main memory (RAM) into fixed-size blocks called Frames. Pages can be stored in any available frames. Page size and frame size are always equal.

Example: A process has 4 pages. These pages can be stored in different free frames in RAM.

2. Paging Hardware

Paging hardware helps the CPU access the correct location in RAM. It mainly uses Page Table, Page Number, Frame Number and Memory Management Unit (MMU).

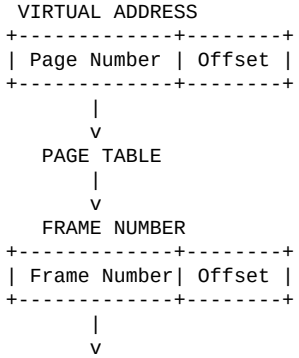


3. Address Translation

Address Translation is the process of converting a Virtual Address into a Physical Address. A virtual address contains a Page Number, which identifies the page, and an Offset, which identifies the exact location inside the page.

Steps

1. CPU generates a virtual address.
2. MMU separates the page number and offset.
3. Page number is checked in the page table.
4. Page table gives the frame number.
5. Frame number is combined with the offset.
6. Physical address is generated.



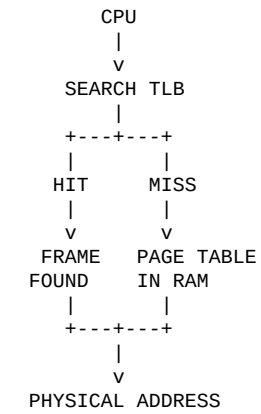
```
      PHYSICAL ADDRESS
```

4. Translation Lookaside Buffer (TLB)

TLB is a small and fast memory that stores recently used page table entries. It helps to translate addresses quickly.

Working

1. CPU generates a virtual address.
2. MMU searches for the page number in TLB.
3. If found, it is called a TLB Hit.
4. If not found, it is called a TLB Miss.
5. During a miss, the page table in RAM is checked.
6. The new entry is added to TLB.



Example: If Page 2 is already available in TLB, the frame number is found quickly without checking the page table in RAM.

5. Demand Paging

Demand Paging is a memory management technique in which pages are loaded into RAM only when they are needed.

Working

1. A program starts execution.
2. CPU requests a particular page.
3. OS checks whether the page is in RAM.
4. If the page is available, execution continues.
5. If the page is missing, a Page Fault occurs.
6. OS loads the required page from disk into RAM.

Example: If a program has 100 pages, but currently needs only 10 pages, only those required pages are loaded.

6. Page Fault Handling

A Page Fault occurs when a process requests a page that is not currently available in RAM.

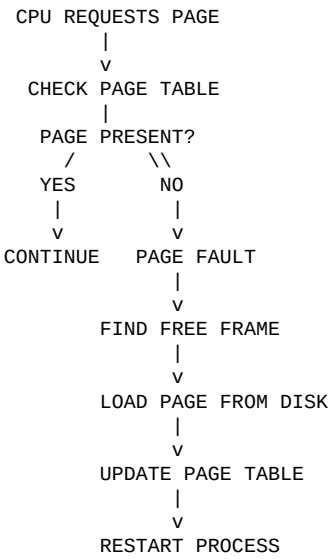
Steps

1. CPU requests a page.
2. OS checks the page table.
3. If the page is missing, a Page Fault occurs.
4. OS checks whether the memory request is valid.
5. OS finds a free frame. If no free frame is available, it selects a page for replacement.

6. Required page is loaded from disk into RAM.

7. Page table is updated.

8. The interrupted instruction is restarted.



7. Advantages

• Paging avoids external fragmentation.

• TLB improves address translation speed.

• Demand paging saves RAM.

• Large programs can run easily.

• Memory is used efficiently.

8. Disadvantages

• Page tables require extra memory.

• Page faults take extra time.

• Too many page faults cause Thrashing.

• Address translation needs additional hardware.

9. Conclusion

Paging divides memory into pages and frames. Address Translation converts virtual addresses into physical addresses. TLB makes translation faster, and Demand Paging loads pages only when required.

Remember: Paging = Pages + Frames. Address Translation = Virtual to Physical. TLB = Fast Page Table. Demand Paging = Load When Needed. Page Fault = Page Not in RAM.

Q2. Explain Mass-Storage Structure, Hard Disk Geometry, and Disk Scheduling Algorithms (FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK) with comparison.

1. What is Mass Storage?

Mass Storage is used to store large amounts of data permanently. It stores data even when the computer is switched OFF.

Examples: Hard Disk (HDD), Solid State Drive (SSD), USB Drive and Memory Card.

2. Hard Disk Structure

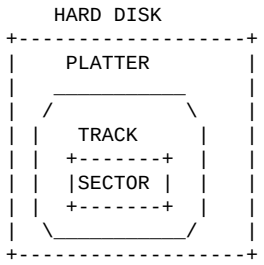
• Platter: Circular disk used to store data.

• Track: Circular path on the platter.

• Sector: Small division of a track.

• Cylinder: Collection of tracks at the same position on different platters.

• Read/Write Head: Used to read and write data.



3. Disk Scheduling

Disk Scheduling is a method used by the Operating System to decide the order of disk requests. Its main purpose is to reduce disk-head movement and waiting time.

4. FCFS (First-Come, First-Served)

FCFS serves disk requests in the order in which they arrive.

1. Requests enter a queue.

2. The disk head takes the first request.

3. It completes that request.

4. It moves to the next request.

5. This continues until all requests are completed.

Example: Head position = 50; Requests = 40, 20, 60. Head Movement: 50 → 40 → 20 → 60.

Advantages and Disadvantage

• Advantages: Simple to understand, easy to implement, no starvation due to request ordering.

• Disadvantage: Head movement may be high.

5. SSTF (Shortest Seek Time First)

SSTF selects the request nearest to the current disk-head position.

1. Check all pending requests.

2. Find the nearest request.

3. Move the head to that request.

4. Remove the completed request.

5. Repeat until all requests are completed.

Example: Head position = 50; Requests = 45, 52, 90. Head Movement: 50 → 52 → 45 → 90. 52 is selected first because it is nearest to 50.

Advantages and Disadvantage

• Advantages: Reduces head movement; better performance than FCFS in many cases.

• Disadvantage: Far requests may wait for a long time.

6. SCAN (Elevator Algorithm)

SCAN moves the disk head in one direction and serves all requests on the way. After reaching the disk end, it reverses direction.

1. Disk head starts at its current position.

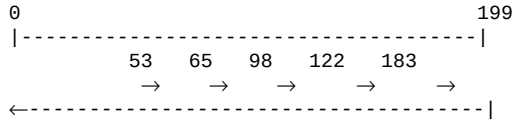
2. It moves in one direction.

3. It serves requests along the way.

4. It reaches the disk end.

5. It reverses direction.

6. It serves remaining requests.



Advantages: Reduces unnecessary movement; provides better waiting time than FCFS in many cases. Disadvantage: It may travel to the disk end even when no request is present there.

7. C-SCAN (Circular SCAN)

C-SCAN moves the disk head in only one direction. After reaching the last cylinder, it returns to the beginning and continues in the same direction.

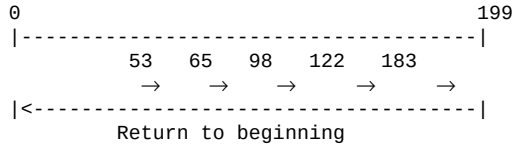
1. Head moves in one direction.

2. It serves all requests along the way.

3. It reaches the last cylinder.

4. It returns to the starting end.

5. It continues serving requests in the same direction.



Advantages: Provides more uniform waiting time; treats requests fairly. Disadvantage: Returning to the beginning requires extra movement.

8. LOOK

LOOK works like SCAN, but the disk head does not travel to the physical end of the disk. It reverses direction at the last pending request.

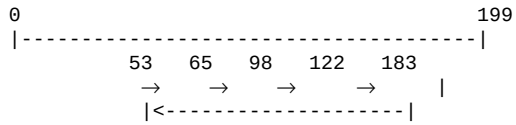
1. Head moves in one direction.

2. It serves requests.

3. It stops at the last request in that direction.

4. It reverses direction.

5. It serves the remaining requests.



Advantages: Avoids unnecessary movement; better efficiency than SCAN in suitable workloads. Disadvantage: Waiting time can vary depending on request positions.

9. C-LOOK (Circular LOOK)

C-LOOK works like C-SCAN, but it moves only up to the last request. Then it jumps to the lowest pending request and continues in the same direction.

1. Head moves in one direction.

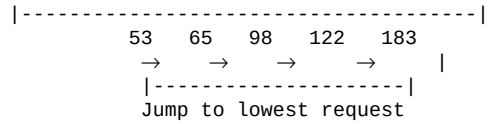
2. It serves all requests.

3. It stops at the highest request.

4. It jumps to the lowest pending request.

5. It continues in the same direction.





Advantages: Avoids unnecessary travel to disk ends; provides more uniform service. Disadvantage: Circular movement requires additional head movement.

10. Comparison of Disk Scheduling Algorithms

Algorithm	Working	Main Advantage
FCFS	Arrival order	Simple and fair
SSTF	Nearest request	Reduces head movement
SCAN	Moves and reverses	Good overall performance
C-SCAN	One direction, circular return	Uniform waiting
LOOK	Reverses at last request	Avoids unnecessary movement
C-LOOK	Jumps between pending requests	Avoids disk-end travel

11. Conclusion

Mass-storage management organizes permanent storage, while disk scheduling improves the order of disk requests and reduces head movement.

Remember: FCFS = Arrival order. SSTF = Nearest request. SCAN = Move and reverse at disk end. C-SCAN = Circular movement. LOOK = Reverse at last request. C-LOOK = Jump between pending requests.

SECTION D – NUMERICAL & ANALYTICAL PROBLEMS

All numerical problems are shown with steps, tables, calculations and final answers.

Problem 1. FIFO (First-In, First-Out) Page Replacement Algorithm

Given Data

• Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

• Number of Frames = 3.

• Initially all frames are empty.

Step 1: Understand FIFO

FIFO removes the page that entered memory first.

Step 2: Fill the Frames

Reference	F1	F2	F3	Result
7	7	–	–	PF
0	7	0	–	PF
1	7	0	1	PF
2	2	0	1	PF
0	2	0	1	H
3	2	3	1	PF
0	2	3	0	PF
4	4	3	0	PF
2	4	2	0	PF
3	4	2	3	PF
0	0	2	3	PF

Reference	F1	F2	F3	Result
3	0	2	3	H
2	0	2	3	H
1	0	1	3	PF
2	0	1	2	PF
0	0	1	2	H
1	0	1	2	H
7	7	1	2	PF
0	7	0	2	PF
1	7	0	1	PF

Step 3: Calculate Hits and Page Faults

• Total References = 20

• Total Hits = 5

• Total Page Faults = 15

Step 4: Calculate Hit Ratio

Hit Ratio = Hits / Total References = 5 / 20 = 0.25 = 25%.

Step 5: Calculate Page Fault Ratio

Page Fault Ratio = 15 / 20 = 0.75 = 75%.

Remember: Hits = 5; Page Faults = 15; Hit Ratio = 25%; Page Fault Ratio = 75%.

Problem 2. LRU (Least Recently Used) Page Replacement Algorithm

Given Data

• Same reference string as Problem 1.

• Number of Frames = 3.

Step 1: Understand LRU

LRU replaces the page that has not been used for the longest time in the past.

Step 2: Solve the Problem

Reference	F1	F2	F3	Result
7	7	–	–	PF
0	7	0	–	PF
1	7	0	1	PF
2	2	0	1	PF
0	2	0	1	H
3	2	0	3	PF
0	2	0	3	H
4	4	0	3	PF
2	4	0	2	PF
3	4	3	2	PF
0	0	3	2	PF
3	0	3	2	H
2	0	3	2	H

Reference	F1	F2	F3	Result
1	1	3	2	PF
2	1	3	2	H
0	1	0	2	PF
1	1	0	2	H
7	1	0	7	PF
0	1	0	7	H
1	1	0	7	H

Step 3: Calculate Results

• Total References = 20

• Hits = 8

• Page Faults = 12

Step 4: Hit Ratio

Hit Ratio = 8 / 20 = 0.40 = 40%.

Step 5: Page Fault Ratio

Page Fault Ratio = 12 / 20 = 0.60 = 60%.

Remember: Hits = 8; Page Faults = 12; Hit Ratio = 40%; Page Fault Ratio = 60%.

Problem 3. Optimal (OPT) Page Replacement Algorithm

Given Data

• Same reference string as Problem 1.

• Number of Frames = 3.

Step 1: Understand Optimal Algorithm

Optimal Page Replacement replaces the page that will not be used for the longest time in the future. If a page will never be used again, it is a good choice for replacement.

Step 2: Solve the Problem

Reference	F1	F2	F3	Result
7	7	–	–	PF
0	7	0	–	PF
1	7	0	1	PF
2	2	0	1	PF
0	2	0	1	H
3	2	0	3	PF
0	2	0	3	H
4	2	4	3	PF
2	2	4	3	H
3	2	4	3	H
0	2	0	3	PF
3	2	0	3	H
2	2	0	3	H
1	2	0	1	PF

Reference	F1	F2	F3	Result
2	2	0	1	H
0	2	0	1	H
1	2	0	1	H
7	7	0	1	PF
0	7	0	1	H
1	7	0	1	H

Step 3: Calculate Results

• Total References = 20

• Hits = 11

• Page Faults = 9

Step 4: Hit Ratio

Hit Ratio = 11 / 20 = 0.55 = 55%.

Step 5: Page Fault Ratio

Page Fault Ratio = 9 / 20 = 0.45 = 45%.

Remember: Hits = 11; Page Faults = 9; Hit Ratio = 55%; Page Fault Ratio = 45%.

Problem 4. Comparative Analysis of FIFO vs LRU vs Optimal

1. What is Page Replacement Comparison?

It means comparing different page replacement algorithms based on their performance.

2. Results

Algorithm	Hits	Page Faults	Hit Ratio
FIFO	5	15	25%
LRU	8	12	40%
Optimal	11	9	55%

3. Explanation

FIFO: Replaces the oldest page. It is simple to implement. Gives 15 Page Faults in this example.

LRU: Replaces the least recently used page. Uses past references. Gives 12 Page Faults.

Optimal: Replaces the page needed farthest in the future. Gives the minimum possible Page Faults. Gives 9 Page Faults.

4. Conclusion

For this reference string, Optimal performs best, followed by LRU and FIFO.

Remember: FIFO = Oldest page. LRU = Least recently used page. OPT = Farthest future use.

Problem 5. Comprehensive Disk Scheduling (FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK)

1. Given Data

• Total cylinders = 200.

• Cylinder range = 0 to 199.

• Initial Head Position = 53.

• Request Queue = 98, 183, 37, 122, 14, 124, 65, 67.

2. Formula

Total Head Movement = Sum of the absolute differences between consecutive head positions.

3. FCFS (First-Come, First-Served)

Rule: Serve requests in arrival order.

```
53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67
```

Calculation: |98−53| + |183−98| + |37−183| + |122−37| + |14−122| + |124−14| + |65−124| + |67−65| = 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = 640 cylinders.

4. SSTF (Shortest Seek Time First)

Rule: Select the nearest request from the current head position.

Starting position = 53. Nearest request = 65 (distance 12). From 65, nearest = 67. From 67, nearest = 37. From 37, nearest = 14. Then serve 98, 122, 124 and 183.

```
53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183
```

Calculation: 12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = 236 cylinders.

5. SCAN Algorithm

Rule: Head moves in one direction, serves requests and reverses at the disk end. The question specifies both directions, so both are shown.

A. Moving Towards 0 First

```
53 → 37 → 14 → 0 → 65 → 67 → 98 → 122 → 124 → 183 → 199
```

Calculation: 16 + 23 + 14 + 65 + 2 + 31 + 24 + 2 + 59 + 16 = 252 cylinders.

B. Moving Towards 199 First

```
53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 37 → 14 → 0
```

Calculation: 12 + 2 + 31 + 24 + 2 + 59 + 16 + 162 + 23 + 14 = 345 cylinders.

6. LOOK Algorithm

Rule: LOOK works like SCAN, but it reverses at the last request instead of going to the disk end.

Moving Towards 0 First:

```
53 → 37 → 14 → 65 → 67 → 98 → 122 → 124 → 183
```

Calculation: 16 + 23 + 51 + 2 + 31 + 24 + 2 + 59 = 208 cylinders.

7. C-SCAN Algorithm

Rule: C-SCAN serves requests in one direction. After reaching the end, it returns to the beginning and continues in the same direction.

Moving Towards 199 First:

```
53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 0 → 14 → 37
```

Calculation: 12 + 2 + 31 + 24 + 2 + 59 + 16 + 199 + 14 + 23 = 382 cylinders.

8. C-LOOK Algorithm

Rule: C-LOOK works like C-SCAN but jumps between the last and first pending requests instead of travelling to the physical disk ends.

Moving Towards 199 First:

```
53 → 65 → 67 → 98 → 122 → 124 → 183 → 14 → 37
```

Calculation: 12 + 2 + 31 + 24 + 2 + 59 + 169 + 23 = 322 cylinders.

9. Final Comparison

Algorithm	Total Head Movement
FCFS	640
SSTF	236

Algorithm	Total Head Movement
SCAN (0 first)	252
SCAN (199 first)	345
LOOK (0 first)	208
C-SCAN (199 first)	382
C-LOOK (199 first)	322

LOOK towards 0 first gives the lowest head movement in the calculated cases: 208 cylinders.

10. Conclusion

Disk Scheduling algorithms decide the order of disk requests. They reduce head movement and improve disk performance.

Remember: FCFS = Arrival order. SSTF = Nearest request. SCAN = Move and reverse at disk end. C-SCAN = Circular movement. LOOK = Reverse at last request. C-LOOK = Jump between pending requests.

Problem 6. Effective Memory Access Time (EMAT) with TLB Analysis

1. What is EMAT?

EMAT stands for Effective Memory Access Time. It is the average time required to access memory when a TLB is used.

2. Given Data

• TLB Access Time = 10 milliseconds.

• Memory Access Time = 80 milliseconds.

• TLB Hit Ratio = 0.6.

3. Step 1 – Calculate Hit Time

When the page entry is found in TLB: Hit Time = TLB + Memory = 10 + 80 = 90 ms.

4. Step 2 – Calculate Miss Time

When the entry is not found in TLB, the page table is checked in RAM. Then actual memory is accessed. Miss Time = TLB + Memory + Memory = 10 + 80 + 80 = 170 ms.

5. Step 3 – Calculate Miss Ratio

Miss Ratio = 1 − Hit Ratio = 1 − 0.6 = 0.4.

6. Step 4 – Apply EMAT Formula

EMAT = (Hit Ratio × Hit Time) + (Miss Ratio × Miss Time)

= (0.6 × 90) + (0.4 × 170) = 54 + 68 = 122 ms.

7. Conclusion

EMAT calculates the average memory access time using TLB Hit Ratio and Miss Ratio. A higher hit ratio generally reduces memory access time.

Remember: EMAT = Hit Time × Hit Ratio + Miss Time × Miss Ratio.

Problem 7. Memory Partition Allocation Strategies (First-Fit, Best-Fit, Worst-Fit)

1. What is Memory Allocation?

Memory Allocation is the process of assigning available memory blocks to different processes. There are three important methods: First-Fit, Best-Fit and Worst-Fit.

2. Given Data

Memory Partitions: 500 KB, 100 KB, 200 KB, 300 KB, 600 KB.

Process	Required Memory
P1	212 KB
P2	417 KB
P3	112 KB
P4	426 KB

3. First-Fit Algorithm

First-Fit selects the first available partition that is large enough for the process.

Working

1. Start checking from the first partition.
2. Find the first suitable partition.
3. Allocate it to the process.
4. Continue with the next process.

Step-by-step: P1 = 212 KB → 500 KB. P2 = 417 KB → 600 KB. P3 = 112 KB → 200 KB. P4 = 426 KB → No suitable partition available.

Process	Size	Allocated Partition	Status
P1	212 KB	500 KB	Allocated
P2	417 KB	600 KB	Allocated
P3	112 KB	200 KB	Allocated
P4	426 KB	–	Not Allocated

4. Best-Fit Algorithm

Best-Fit selects the smallest available partition that is large enough for the process.

Working

1. Check all available partitions.
2. Find the smallest suitable partition.
3. Allocate it to the process.
4. Repeat for the next process.

Step-by-step: P1 = 212 KB → 300 KB. P2 = 417 KB → 500 KB. P3 = 112 KB → 200 KB. P4 = 426 KB → 600 KB.

Process	Size	Allocated Partition	Status
P1	212 KB	300 KB	Allocated
P2	417 KB	500 KB	Allocated
P3	112 KB	200 KB	Allocated
P4	426 KB	600 KB	Allocated

5. Worst-Fit Algorithm

Worst-Fit selects the largest available partition for the process.

Working

1. Check all available partitions.
2. Find the largest suitable partition.
3. Allocate it to the process.
4. Repeat for the next process.

Step-by-step: P1 = 212 KB → 600 KB. P2 = 417 KB → 500 KB. P3 = 112 KB → 300 KB. P4 = 426 KB → No suitable partition available.

Process	Size	Allocated Partition	Status
P1	212 KB	600 KB	Allocated
P2	417 KB	500 KB	Allocated
P3	112 KB	300 KB	Allocated
P4	426 KB	–	Not Allocated

6. Final Comparison

Algorithm	Processes Allocated	Not Allocated
First-Fit	3	1
Best-Fit	4	0
Worst-Fit	3	1

7. Which Method Is Best?

For this particular example, Best-Fit allocates all four processes successfully. Therefore, Best-Fit is the most successful method for the given partition sizes and process order.

8. Conclusion

First-Fit selects the first suitable block, Best-Fit selects the smallest suitable block, and Worst-Fit selects the largest suitable block.

Remember: First-Fit = First suitable. Best-Fit = Smallest suitable. Worst-Fit = Largest suitable.